

MÔ TẢ SEQUENCE DIAGRAM CỦA HỆ THỐNG GAME (EVENT-DRIVEN)

1. Tổng quan Sequence Diagram

Mọi tương tác của người dùng, sự kiện thời gian và logic trò chơi đều được truyền thông qua Event Ring Buffer (EventRB).

Các thành phần chính tham gia trong các sequence bao gồm:

- **User**: Người chơi thao tác nút nhấn
- **Button**: Phát sinh sự kiện nút nhấn
- **EventRB**: Hàng đợi sự kiện trung tâm
- **GameAO**: Active Object điều phối trạng thái game
- **Game**: Xử lý logic gameplay
- **Signal_handle**: Điều khiển LED và buzzer
- **Render**: Vẽ giao diện lên OLED

2. Sequence: Vào game và điều khiển nhân vật

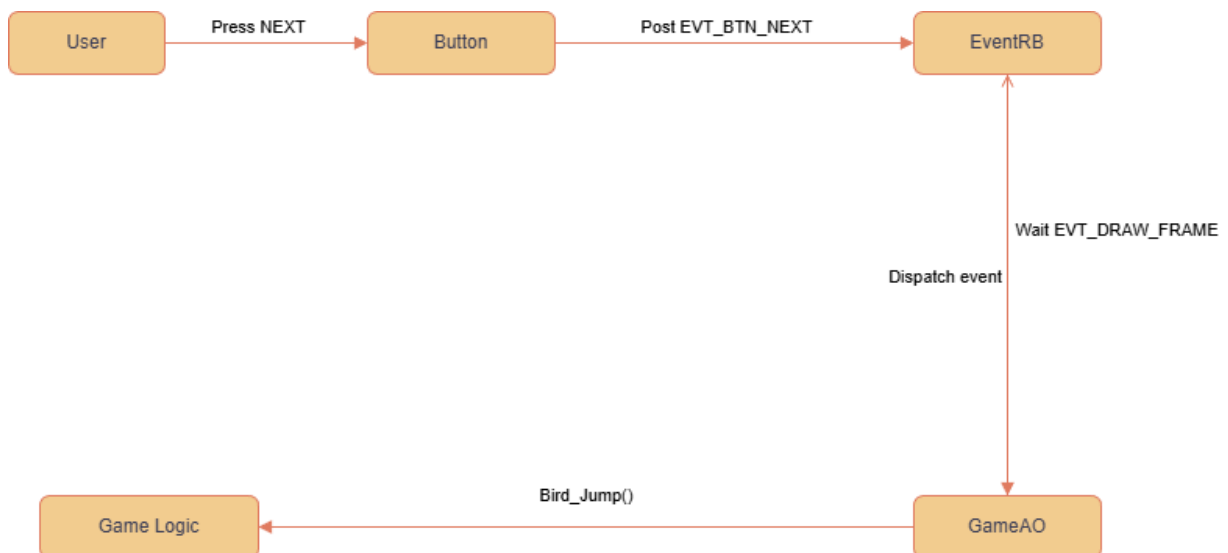
Mô tả

Khi người chơi nhấn nút NEXT, module Button tạo sự kiện EVT_BTN_NEXT và đưa vào EventRB.

GameAO nhận sự kiện và kiểm tra trạng thái hiện tại của game.

- Nếu game đang ở trạng thái GAME_PLAYING, GameAO gọi hàm Bird_Jump() để cập nhật vận tốc của nhân vật.
- Không có xử lý trực tiếp trong main loop, mọi logic đều được kích hoạt bởi sự kiện.

Sau khi xử lý, hệ thống không vẽ ngay mà chờ sự kiện vẽ khung hình tiếp theo.



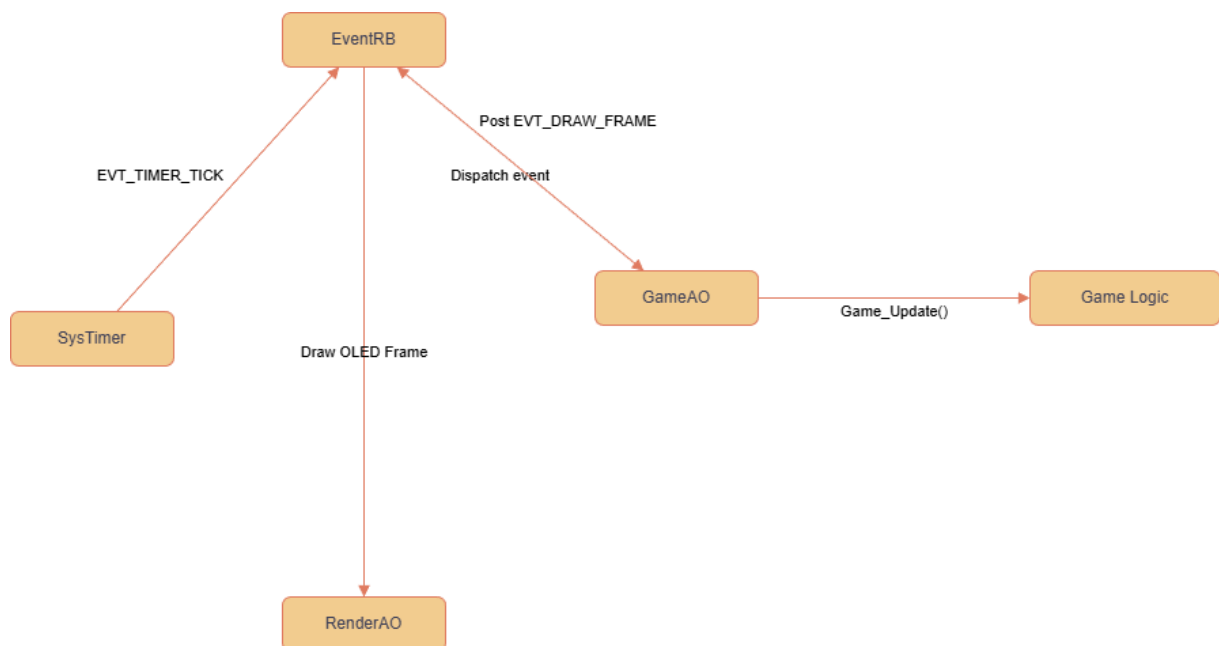
3. Sequence: Cập nhật game theo Timer (Gameplay Loop)

Mô tả

Định kỳ, timer hệ thống phát sinh sự kiện EVT_TIMER_TICK và đưa vào EventRB. Khi GameAO nhận được sự kiện này trong trạng thái GAME_PLAYING, nó thực hiện các bước:

1. Gọi hàm Game_Update() để:
 - Cập nhật vị trí chim
 - Cập nhật vị trí ống
 - Kiểm tra va chạm
 - Cập nhật điểm số
2. Sau khi xử lý logic xong, GameAO gửi sự kiện EVT_DRAW_FRAME để yêu cầu vẽ lại màn hình.

Module Render nhận sự kiện và vẽ frame mới lên màn hình OLED.



4. Sequence: Phát hiện Game Over

Mô tả

Trong quá trình thực thi Game_Update(), nếu xảy ra một trong các điều kiện sau:

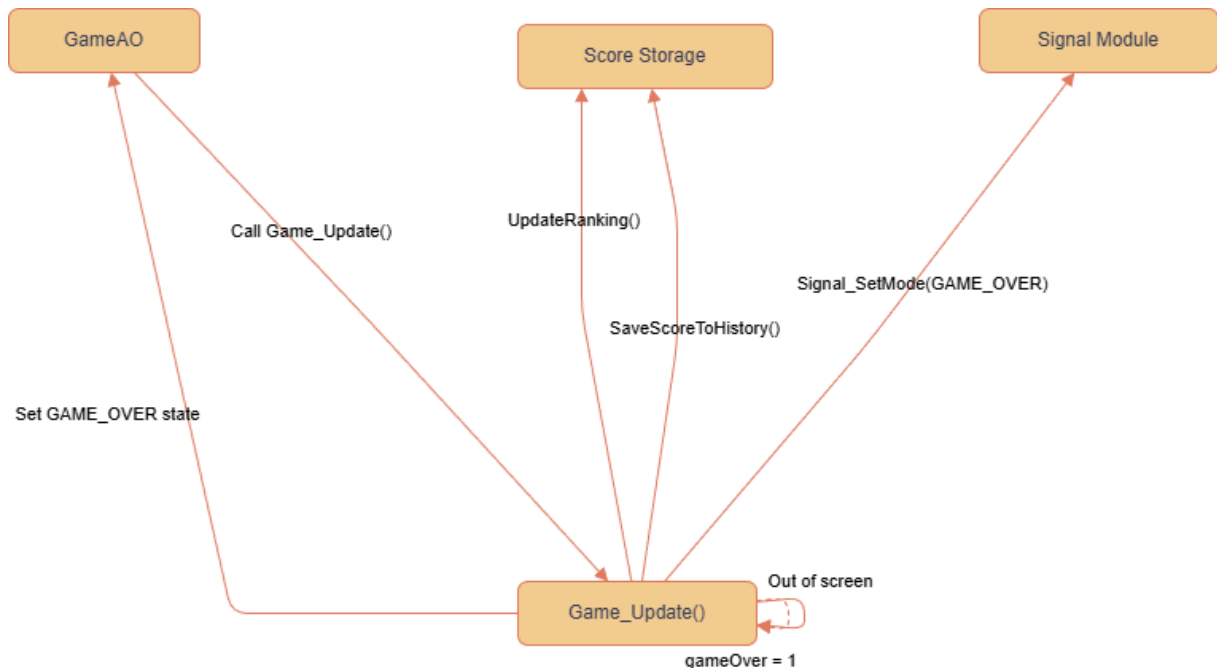
- Chim va chạm ống
- Chim rơi ra khỏi màn hình
- Chim va chạm khe trong chế độ đặc biệt

Hệ thống thực hiện các hành động theo thứ tự:

1. Đặt cờ gameOver = 1 để dừng cập nhật game
2. Lưu điểm số vào lịch sử (SaveScoreToHistory)
3. Cập nhật bảng xếp hạng (UpdateRanking)

4. Chuyển trạng thái hệ thống sang GAME_OVER
5. Kích hoạt tín hiệu cảnh báo bằng cách gọi
Signal_SetMode(SIGNAL_MODE_GAME_OVER)

Sau đó, hàm Game_Update() kết thúc và không tiếp tục xử lý game.



Ý nghĩa thiết kế

- Game chỉ phát hiện trạng thái, không điều khiển phần cứng
- Trách nhiệm được phân tách rõ ràng
- Dễ thay đổi hành vi khi Game Over mà không sửa logic game

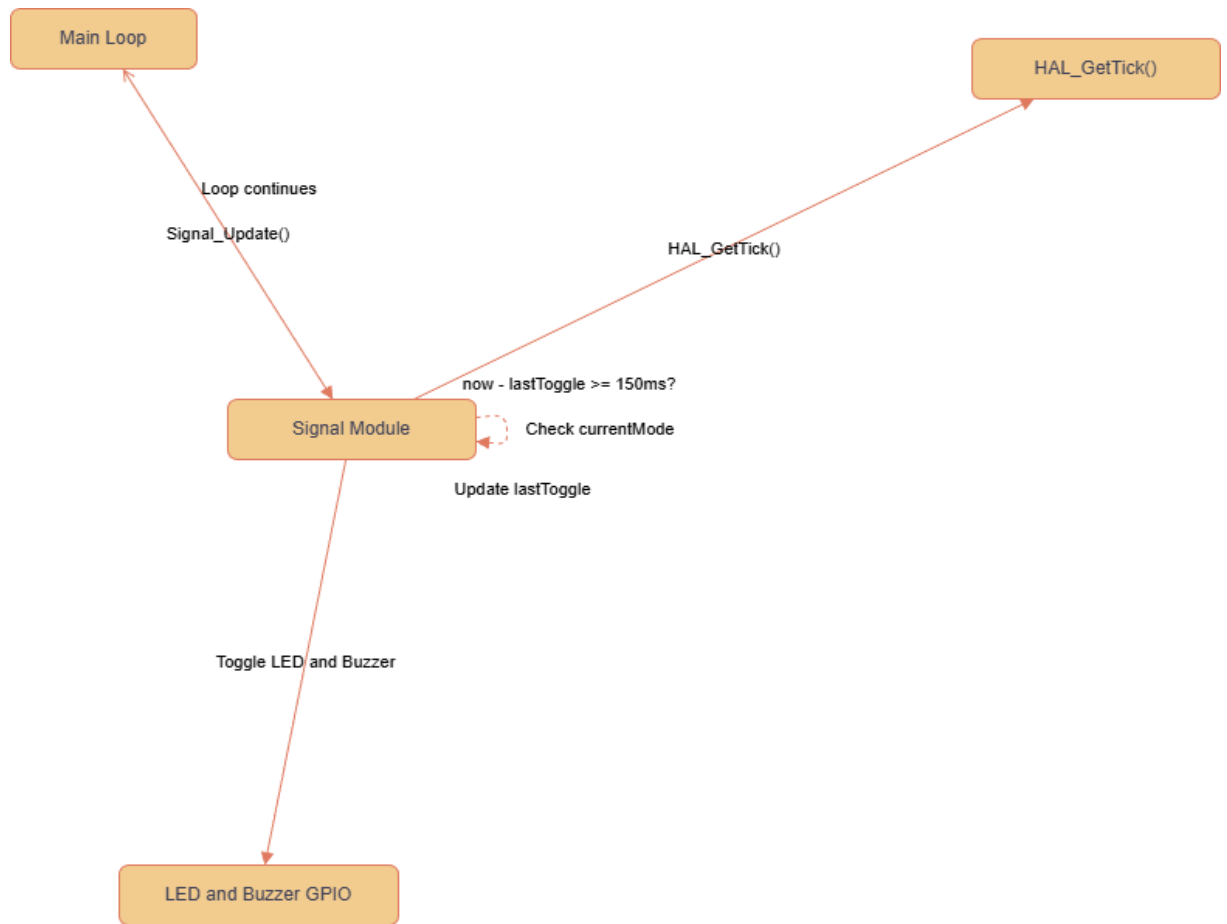
5. Sequence: Điều khiển LED và Buzzer khi Game Over

Mô tả

Sau khi chế độ SIGNAL_MODE_GAME_OVER được kích hoạt, module Signal hoạt động độc lập trong vòng lặp chính của hệ thống.

Tại mỗi lần gọi Signal_Update():

1. Module Signal kiểm tra mode hiện tại
2. Nếu mode là GAME_OVER:
 - Kiểm tra thời gian bằng HAL_GetTick()
 - Định kỳ mỗi 150ms, đảo trạng thái LED và buzzer
3. Quá trình này lặp lại liên tục cho đến khi có lệnh dừng



6. Sequence: Thoát Game Over và quay về Home

Mô tả

Khi game đang ở trạng thái GAME_OVER, nếu người chơi nhấn nút **MENU**:

1. Button phát sinh sự kiện EVT_BTN_MENU
2. GameAO nhận sự kiện và thực hiện:
 - Gọi Signal_StopAll() để tắt LED và buzzer
 - Gọi Game_Init() để reset toàn bộ game
 - Chuyển trạng thái sang GAME_HOME
 - Phát sự kiện EVT_DRAW_FRAME để vẽ lại giao diện Home

Màn hình Home được hiển thị và hệ thống sẵn sàng cho lượt chơi mới.

